Comparative Evaluation of Metric-Based and Signature-Based Detection in a Containerized Network Observability and Intrusion Alerting Stack

Computer Networks - Term Project Report

BS CS, 6th Semester, Laar Campus Badin

Instructor: Dileep Kumar

Team member	Student ID	Email
Maqsood Ahmed	2K24/BLCS/31	2k24-blcs-31@students.usindh.edu.pk
Aijaz Ali	2K24/BLCS/6	2k24-blcs-6@students.usindh.edu.pk

Repository: https://github.com/maqsoodbhurgri/Networking_Project

Abstract

Small networks often notice port scans, floods and service failures only after users are affected. We built a four-zone network lab from Docker containers, with a Linux router enforcing a default-deny nftables policy, and deployed it as code with Docker Compose. We then compared two detection methods on the same traffic: Method A, metric/threshold rules in Prometheus on router packet counters, and Method B, signature rules in Suricata. Each experiment was repeated ten times. Suricata detected all four attack types (port scan, SYN flood, ICMP flood, SSH brute force) in 0.13 to 1.44 s on average. Prometheus detected the first three in about 8 to 11 s and missed the SSH brute force in all runs, because its traffic stayed below the packet-rate threshold. A manual tcpdump baseline took a median of about 54 s. Both methods produced zero false alerts in 30 minutes of normal traffic. Service-down faults were detected in 4.8 s on average. Monitoring added no measurable overhead at a fixed 1 Gbps load, and only Suricata's CPU use grew with load. All results come from one laptop, so they describe this emulated environment and should not be read as production figures.

1. Introduction

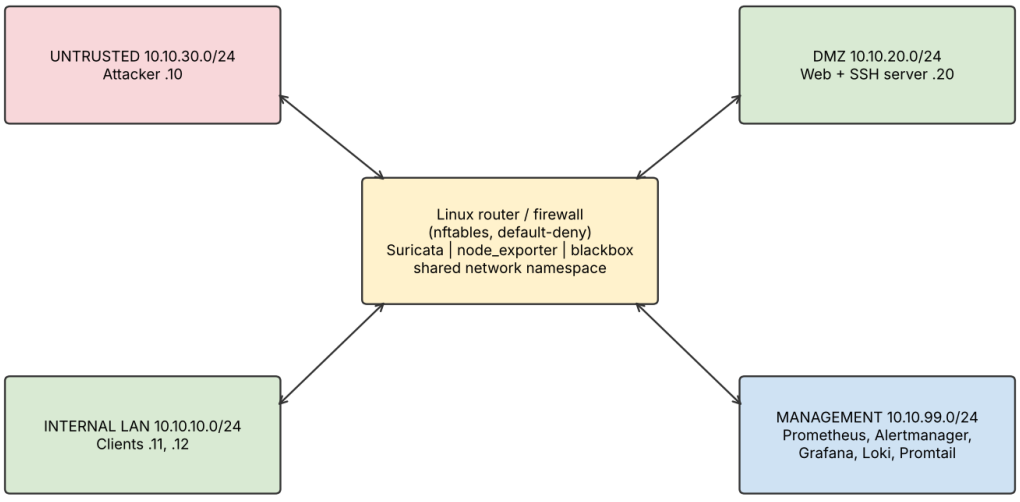
Problem. Port scans, SYN floods and service failures are often found late in small and medium networks. Late detection prolongs downtime and gives an attacker time to probe and escalate. Monitoring tools exist, but it is rarely measured which detection method reacts fastest and at what cost. Monitoring stacks are also frequently built by hand, which makes them hard to reproduce.

Contribution. (i) A containerized observability and alerting stack, deployed as code and versioned in Git, with a CI job that lints its configuration. (ii) A comparative evaluation of Method A (Prometheus rules on router counters) and Method B (Suricata signatures) on detection latency, detection rate, false positives and resource overhead, against a manual baseline.

Scope. Implementation paradigm A (network emulation and virtualization); primary domain E (network observability); secondary domain D (zone segmentation and intrusion detection). Out of scope: automatic blocking, machine learning, INT/P4, physical hardware, encrypted payload inspection, IPv6. All attack traffic stays inside the isolated lab.

2. Architecture

Figure 1: Logical topology and trust zones



A Linux router joins four zones: untrusted (10.10.30.0/24), internal LAN (10.10.10.0/24), DMZ (10.10.20.0/24) and management (10.10.99.0/24). The router is a container with IP forwarding enabled. Suricata, node_exporter and the blackbox exporter run in the router's network namespace, so every inter-zone packet is visible without a mirror port.

Suricata is attached to the untrusted-facing interface only, because attaching it to all interfaces would show each forwarded packet twice and produce duplicate alerts. The consequence is that traffic between the LAN and the DMZ is not inspected by Suricata.

Docker assigns interface names (eth0 to eth3) in no fixed order, and they changed after restarts. A start script, start_lab.sh, detects the interface that carries 10.10.30.1 and rewrites the Suricata and Prometheus settings accordingly.

2.1 Firewall policy (default-deny)

From	To	Allowed
Untrusted	DMZ web server (.20)	TCP 22 and 80, ICMP echo request
LAN	DMZ web server (.20)	TCP 22, 80, 5201; UDP 5201; ICMP echo request
Management	All zones	All traffic (telemetry is initiated from inside it)
Everything else		Counted and dropped

The policy was verified by test: the attacker could reach the web server on ports 22 and 80, pings between the untrusted zone and the LAN failed with 100% loss, and an nmap scan of ports 22, 80 and 5201 showed 5201 as filtered. The drop counter rose during the tests.

3. Implementation

Component	Role
Docker Compose	Deploys 13 containers: router, attacker, client1, client2, web, node_exporter, Suricata, Prometheus, Alertmanager, Grafana, Loki, Promtail, blackbox exporter
nftables (router)	Default-deny forwarding policy between zones
Prometheus + node_exporter	Scrapes router counters every 5 s; evaluates alert rules every 5 s (Method A)
Suricata 8.0.8	Signature IDS on the untrusted interface (Method B); writes eve.json
Blackbox exporter	HTTP probe of the DMZ web server every 5 s for the service-down alert
Alertmanager, Grafana	Alert routing and dashboards (Node Exporter Full, imported from Grafana.com, ID 1860)
Loki + Promtail	Log pipeline reading Suricata's eve.json
GitHub Actions	Lints YAML, validates the Compose file, runs promtool on Prometheus config and rules, checks Suricata rule IDs are unique

3.1 Detection rules

Method B (Suricata) uses four threshold signatures on traffic from the untrusted zone: port scan (20 SYNs from one source within 5 s, sid 1000001), SYN flood (1000 SYNs within 1 s, sid 1000002), ICMP flood (50 echo requests within 1 s, sid 1000003) and SSH brute force (5 SYNs to port 22 within 30 s, sid 1000004).

Method A (Prometheus) uses two rules on the receive packet rate of the untrusted interface, computed over a 10 s window: SuspiciousPacketRate (above 50 packets per second) and FloodPacketRate (above 5000). A third rule, ServiceDown, fires when the blackbox probe of the web server fails.

3.2 Division of work

Maqsood Ahmed: topology and router, Prometheus/Grafana/Loki stack, E1 and E5. Aijaz Ali: Suricata and alert rules, attack and fault scripts, E2 to E4. Both: CI pipeline, result analysis, report and demo.

4. Methodology

Each experiment was repeated ten times (E5 sampled each load step three times) and reported as mean and standard deviation, unless stated. Scripts are in the scripts folder and raw results are in results as CSV files.

Detection latency is measured from the attack start time to the first alert. For Method B it is the Suricata alert timestamp in eve.json. For Method A it is the first time the alert appears as firing in a Prometheus range query at 1 s resolution, so Method A times are accurate to about plus or minus 1 s. Both methods use the same host clock. The scripts run each attack from the attacker container against the DMZ web server and wait between runs so alert states reset.

Attack	Tool and parameters
Port scan	nmap -sS -Pn -T4 -p 1-2000 --max-retries 1
SYN flood	hping3 -S --flood -p 80, 5 s
ICMP flood	hping3 --icmp --flood, 5 s
SSH brute force	hydra, 8 passwords, 4 tasks

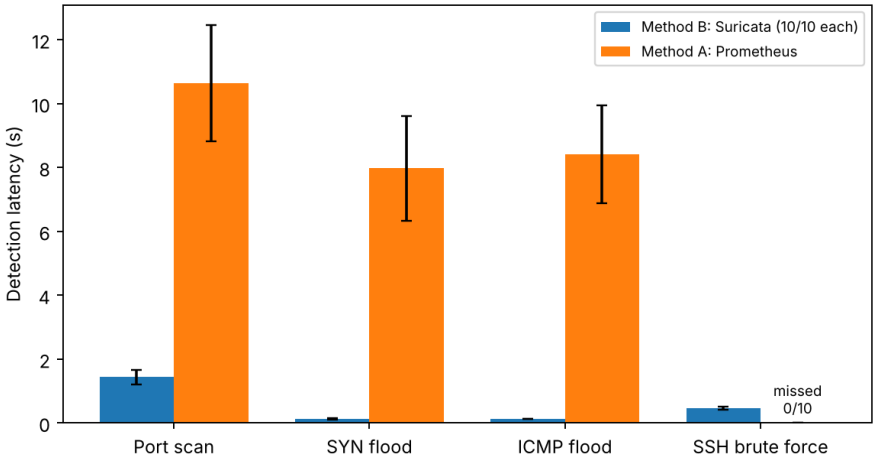
Baselines. The overhead baseline is the same lab with the monitoring containers stopped. The detection baseline is manual: an operator watches filtered tcpdump output and presses a key on spotting a port scan.

5. Results

5.1 E2: attack detection (Method A vs Method B)

Attack	Suricata (B)	Prometheus (A)
Port scan	1.44 +/- 0.23 s, 10/10	10.65 +/- 1.82 s, 10/10
SYN flood	0.13 +/- 0.02 s, 10/10	7.98 +/- 1.64 s, 10/10
ICMP flood	0.13 +/- 0.01 s, 10/10	8.42 +/- 1.53 s, 10/10
SSH brute force	0.46 +/- 0.05 s, 10/10	missed, 0/10

Figure 2: E2 detection latency (mean, sd)



Suricata detected every run of every attack. Prometheus detected 30 of 40 runs (75%), missing all ten SSH brute-force runs. Hydra sends far fewer than 50 packets per second, so the rate threshold was never crossed. Metric-based detection therefore misses low-rate attacks that a signature catches. Prometheus latency of 8 to 11 s comes from its 5 s scrape interval, 5 s evaluation interval and 10 s rate window. The port scan result (10.65 s) is slightly above the 10 s target. A nmap scan also triggered the SYN-flood signature in one earlier observation, so the scan and flood rules can overlap. The ICMP flood and SYN flood reached Prometheus through the same packet-rate rule, since the rule counts all packets on the interface; a real deployment would use per-attack rules.

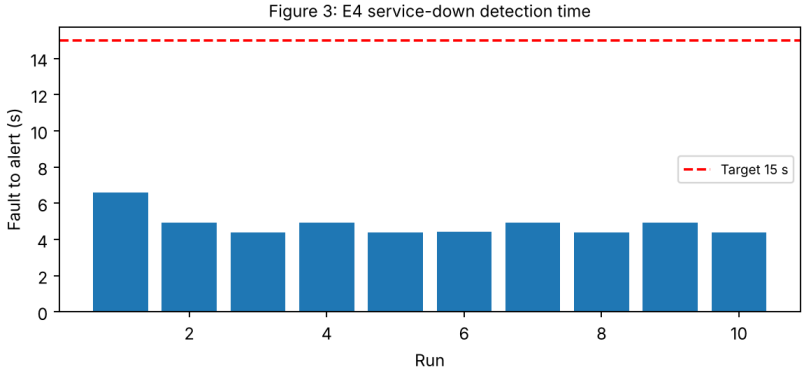
5.2 E3: false positives

Normal traffic (web requests and pings from the untrusted zone and the LAN, with occasional SSH port checks) ran for 30 minutes. Result: 0 false alerts from Suricata and 0 from Prometheus, within the target of 2 or fewer. Before the realistic run, a 60 s test that opened an SSH connection every 3 to 7 s produced 2 Suricata alerts from the SSH brute-force signature and none from Prometheus. A fixed-threshold signature cannot tell frequent legitimate connections from a brute force. It can be tuned, at the risk of missing slower attacks. The 0 and 0 result applies only to this light workload; heavy legitimate traffic, such as a large download, could trigger a packet-rate threshold and was not tested.

5.3 E4: fault detection

The web container was frozen with docker pause, so the blackbox probe timed out, which models a hung service. Detection by the ServiceDown rule took 4.84 +/- 0.67 s (10/10; range 4.39 to 6.60 s), within the 15 s target. Polling the alert API every 0.5 s adds a small measurement error. A link-down fault was not tested.

Figure 3: E4 service-down detection time



5.4 E1: monitoring overhead

An unlimited-speed TCP test was unusable on one shared laptop: throughput jumped between about 4.6 and 14 Gbps whether monitoring was on or off, even with the iperf3 client and server pinned to fixed CPUs. E1 therefore uses a fixed 1 Gbps load, alternating monitoring OFF and ON in ten rounds. Values below are means over 10 runs each (approximate to the digits shown).

Mode	TCP (Mbps)	UDP jitter (ms)	UDP loss (%)	Ping (ms)
Monitoring OFF	992	0.0036	0.042	0.127
Monitoring ON	992	0.0038	0.055	0.126

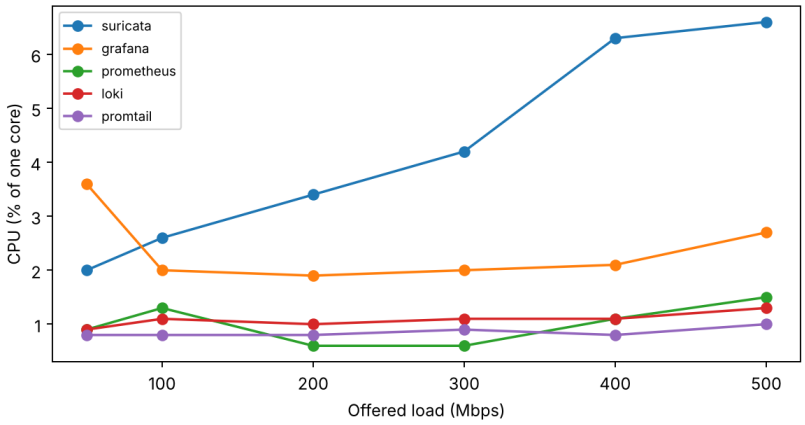
No overhead was measurable: throughput was identical and latency, jitter and loss were the same within noise, against a target of under 10%. Limits: the load is fixed at 1 Gbps, everything shares one CPU, and Suricata watches only the untrusted-facing interface, so this LAN-to-DMZ traffic was not inspected by it. The CPU cost of the stack is measured in E5.

5.5 E5: scaling cost

Attack-free load from the attacker container (untrusted zone) was raised from 50 to 500 Mbps. At 500 Mbps the iperf3 receiver measured 496 Mbps, so the offered rates were reached. Average CPU (percent of one core) per container:

Container	50	100	200	300	400	500 Mbps	Memory
Suricata	2.0	2.6	3.4	4.2	6.3	6.6	42-45 MB
Grafana	3.6	2.0	1.9	2.0	2.1	2.7	about 237 MB
Prometheus	0.9	1.3	0.6	0.6	1.1	1.5	57-58 MB
Loki	0.9	1.1	1.0	1.1	1.1	1.3	42-43 MB
Promtail	0.8	0.8	0.8	0.9	0.8	1.0	18 MB

Figure 4: E5 per-container CPU vs load



Only Suricata's CPU grew with load (about threefold, with flat memory); the rest of the stack stayed near 0 to 4% CPU, and Suricata reported zero kernel packet drops at every load step. Caveat: veth interfaces use offloading, so Suricata sees fewer, larger packets than on a physical wire, and the per-packet load is lighter than in a real network. The router container itself showed about 0% CPU because forwarding happens in the kernel.

5.6 Manual detection baseline

Run	1	2	3	4	5
Time to detect (s)	95.4	147.6	44.9	53.8	12.3

The mean was 70.8 s, the standard deviation 52.2 s and the median 53.8 s (range 12.3 to 147.6 s). The scan itself lasts about 14 s, so in four of five runs it had already ended when the operator reacted. Caveats: only five runs were made, the operator knew an attack was coming, raw packet lines are hard to read, and manual detection works only while someone is actively watching. The numbers are indicative only. All five runs are reported.

5.7 Summary against the proposal targets

Metric	Target	Result	Met?
Detection latency	< 10 s	Suricata 0.13-1.44 s; Prometheus 7.98-10.65 s	Suricata yes; Prometheus floods yes, scan no (10.65 s)
Detection rate	>= 90%	Suricata 40/40; Prometheus 30/40 (75%)	Suricata yes; Prometheus no (SSH missed)
False positives	<= 2 per 30 min	0 and 0 (realistic traffic)	Yes

Metric	Target	Result	Met?
Throughput/latency overhead	< 10%	No measurable difference at 1 Gbps	Yes
CPU / memory	Documented	Table in section 5.5	Yes
Fault detection	< 15 s	4.84 s mean	Yes

6. Discussion

Where each method wins. Suricata was faster on every attack and was the only method to detect the SSH brute force, because its signatures match specific traffic patterns with short time windows. Prometheus was slower by design: a 5 s scrape interval, 5 s rule evaluation and a 10 s rate window set a floor of several seconds. It detects only what changes aggregate counters, so it misses low-rate attacks, and it cannot tell a scan from a flood or from a busy legitimate client. Metric rules are the natural tool for availability events: the same stack detected a failed web service in under 5 s and costs almost nothing to run. The two methods complement each other.

Trade-offs. Tightening Prometheus (shorter scrape interval and window) would reduce latency at the cost of CPU and noisier alerts; this was not tested. Suricata's fixed thresholds can misfire on legitimate bursts, as the frequent-SSH test showed, and tuning them trades false positives against missed slow attacks.

Reproducibility. The whole lab is deployed from a Compose file and scripts in Git, and a green CI job lints the configuration. The detail that made reproduction fragile was Docker's changing interface names, handled by the start script.

Limitations. Everything ran on one laptop with shared CPU, so absolute numbers are environment-specific. Not implemented from the proposal: softflowd (NetFlow export), ntopng (optional) and a link-down fault test. The manual baseline had five runs. Suricata inspects only the untrusted-facing interface. Attack tests used one attacker, one target and one set of tool parameters. The E3 result covers light traffic only.

7. Conclusion and future work

We built a reproducible, segmented, containerized lab and measured two detection methods against the same attacks. Signature-based detection was faster and caught a low-rate attack that metric-based detection missed, and neither method raised false alerts on light normal traffic. Metric-based rules were effective for service-down detection and are the cheaper way to watch availability. Monitoring added no measurable overhead at 1 Gbps, and only Suricata's CPU grew with load. Future work: add NetFlow export and per-attack Prometheus rules, test link-down faults and heavier legitimate traffic, tune the thresholds and shorten the Prometheus intervals, and extend to automatic response.

Appendix: repository contents

Item	Location
Compose file, router firewall	docker-compose.yml, router/nftables.conf
Suricata and Prometheus rules	suricata/local.rules, monitoring/prometheus/rules/lab.yml
Experiment scripts	scripts/ (e1 to e5, baseline, start_lab.sh, make_graphs.py)
Raw results and graphs	results/ and results/graphs/
Grafana dashboard JSON	grafana/node_exporter_dashboard.json
CI lint job	.github/workflows/lint.yml

Item	Location
Sample packet capture	port_scan_sample.pcap (kept locally, not in Git)